
GramVest

SIH 2026 • PS 26091

M5 - AI/RAG, Risk Explanation, Advisor & DPR

Module implementation handbook - AI / RAG owner

TEAM IMPLEMENTATION HANDBOOK

Prepared as a practical engineering guide for independent module development, stable integration, testing, and handoff.

Source basis: SIH 26091 Implementation Blueprint and the corresponding GramVest module
Markdown handbook.

Contents

1	1. Mission	1
2	2. Golden rule	1
3	3. Recommended tools	1
4	4. Repository	1
5	5. RAG knowledge sources	2
6	6. RAG ingestion workflow	2
7	7. Chunking	2
8	8. Retrieval workflow	3
9	9. Intent detection	3
10	10. Structured context assembly	3
11	11. Grounded answer format	3
12	12. Numerical guardrail	4
13	13. Scheme guardrails	4
14	14. SWOT generation	4
15	15. Risk engine	5
16	16. Business recommendation	5
17	17. Multilingual support	5
18	18. Advisor endpoint	6
19	19. DPR generation	6
20	20. DPR workflow	6
21	21. Report templates	7
22	22. AI evaluation set	7
	22.1 Supported	7
	22.2 Unsupported	7
	22.3 Missing-data	7
	22.4 Conflicting documents	7
	22.5 Numerical hallucination	7
23	23. Source handling	7
24	24. Integration with M3/M4	7
25	25. Definition of done	8
26	26. Common mistakes	8

1. Mission

Build the language/intelligence layer that explains structured GramVest outputs without replacing deterministic software.

M5 owns:

- RAG knowledge base
- intent detection
- prompt design
- grounded AI advisor
- multilingual response handling where included
- SWOT narrative
- risk explanations
- recommendation explanations
- hallucination guardrails
- source/version metadata
- DPR narrative and PDF orchestration

2. Golden rule

AI explains; software calculates.

The blueprint assigns:

- conversation/Q&A → LLM
- scheme knowledge → RAG over versioned official documents
- SWOT narrative → LLM using structured evidence
- risk explanation → rules + evidence + LLM
- business recommendation → scoring engine + evidence + LLM
- loan/EMI/DSCR → deterministic financial engine
- scheme routing → deterministic rule engine
- DPR narrative → template + calculations + LLM polishing

3. Recommended tools

Task	Tool
Language	Python
LLM	agreed hosted/local model
Vector search	pgvector or FAISS
Data prep	pandas
Document parsing	Python document/PDF parsing libraries selected by team
PDF	ReportLab
API	FastAPI through M2
Evaluation	pytest + curated question set
API testing	Postman
Version control	Git + GitHub

The blueprint recommends pgvector/FAISS and ReportLab.

4. Repository

ai/
├─ prompts/

```
├─ rag/  
│   ├── ingestion/  
│   ├── chunking/  
│   ├── retrieval/  
│   └─ metadata/  
├─ guardrails/  
├─ advisor/  
├─ risk/  
├─ swot/  
└─ evaluation/  
  
backend/services/  
└─ report/  
    ├── templates/  
    └─ generator.py
```

5. RAG knowledge sources

The blueprint specifically recommends versioned official documents for scheme knowledge.

Store metadata:

```
document_id  
title  
source  
source_url/reference  
issued_date  
effective_date  
version  
jurisdiction  
category
```

Do not ingest random unverified blogs into the scheme knowledge base and present them as official policy.

6. RAG ingestion workflow

```
Official document  
→ extract text  
→ clean  
→ section-aware chunk  
→ attach metadata  
→ embed  
→ store in vector index
```

Keep original document reference for citation.

7. Chunking

Prefer meaningful chunks:

- eligibility
- funding limits
- interest/tenure
- required documents
- application process

Do not split every arbitrary N characters if section structure is available.

8. Retrieval workflow

The blueprint describes:

```
User question
→ Intent detection
→ Retrieve relevant documents
→ Add structured user/market/financial context
→ LLM
→ Grounded answer with source/version
```

Implement this flow explicitly.

9. Intent detection

Start simple.

Possible intents:

```
scheme_question
financial_explanation
market_question
competitor_question
risk_question
business_recommendation
what_if_question
report_question
unsupported
```

A simple classifier/rules layer is acceptable for prototype routing.

10. Structured context assembly

Create one canonical context object:

```
{
  "profile": {},
  "location": {},
  "market": {},
  "competitors": {},
  "opportunity": {},
  "risks": {},
  "viability": {},
  "finance": {},
  "scheme": {},
  "scenario": {}
}
```

Do not ask the LLM to infer financial numbers that already exist in this object.

11. Grounded answer format

Prompt the model to answer in this order:

1. short answer
2. why
3. verified numbers/evidence
4. assumptions
5. source/version if applicable
6. caveat when data is incomplete

Example:

Short answer:

The 15% feed-cost increase still leaves the current scenario above the modeled repayment requirement.

Why:

Your monthly operating surplus falls from X to Y.

Verified values:

...

Assumption:

Feed price changes only; other costs remain unchanged.

Source/version:

...

12. Numerical guardrail

The blueprint requires numerical claims returned by AI to link to structured backend values.

Implement a guardrail that checks whether numeric values in the AI answer are present in the allowed context or generated from explicitly allowed source values.

If the AI invents unsupported numbers:

- reject/regenerate
- or replace with a safe message

If data is missing:

"I do not have enough verified information to estimate that reliably."

13. Scheme guardrails

Always show:

- indicative status
- source/version
- effective date where available

Never let the AI say:

- approved
- guaranteed
- officially eligible

unless the system has a verified official status for that exact user/application and the project scope actually supports it.

The blueprint explicitly says final eligibility/sanction remains with the relevant authority/bank.

14. SWOT generation

Input:

- market evidence
- finance evidence
- competitor evidence
- business profile
- risk evidence

Generate structured:

```
strengths[]  
weaknesses[]  
opportunities[]  
threats[]
```

Every item should contain evidence or an explicit assumption.

15. Risk engine

Risk categories from the blueprint:

- demand
- competition
- supply/input cost
- operational
- seasonal
- financial
- regulatory

Each high-severity risk gets:

```
risk  
severity  
why_it_matters  
evidence  
mitigation
```

Do not let the LLM freely invent risks with no supporting evidence.

16. Business recommendation

The recommendation should be generated from:

```
score  
+  
evidence  
+  
financial results  
+  
market results  
+  
risk results
```

Example structure:

Recommendation:

Promising, with caution on input costs.

Why:

- market demand is healthy
- competition is moderate
- projected cash surplus is positive
- input-cost sensitivity is material

17. Multilingual support

The blueprint allows language detection + LLM/translation and optional STT/TTS.

For MVP, prioritize:

- English
- Hindi

- Punjabi

Make structured financial values language-independent and translate explanatory text.

Do not duplicate calculation logic by language.

18. Advisor endpoint

M2 exposes:

POST /api/advisor/chat

M5 implements the internal advisor service behind that endpoint.

Request should include:

```
user message
profile/context id
language
optional scenario id
```

Response:

```
{
  "answer": "...",
  "evidence": [],
  "numbers": [],
  "sources": [],
  "assumptions": [],
  "confidence": 0.84
}
```

19. DPR generation

The blueprint's report should include:

- executive summary
- entrepreneur profile/assumptions
- location and market reach
- customer/competitor analysis
- opportunity/pricing
- SWOT
- risk register/mitigation
- project cost/means of finance
- revenue/expense assumptions
- projected P&L
- cash flow
- loan/repayment
- DSCR/break-even
- indicative scheme route
- implementation plan
- data sources/dates/confidence
- disclaimer

20. DPR workflow

Get canonical BusinessAnalysisContext

```
→ validate required sections
→ render numeric tables from verified values
→ generate narrative from structured evidence
→ insert source/version notes
```

- generate PDF
- save version metadata

The LLM should polish narrative text only. Numeric tables should come from trusted values.

21. Report templates

Recommended:

```
reports/templates/  
├─ feasibility_summary  
└─ detailed_dpr
```

Use ReportLab as specified by the blueprint.

22. AI evaluation set

Create a curated test set including:

Supported

- "Why is my business score 74?"
- "What happens if feed costs increase 15%?"
- "Which scheme may fit?"

Unsupported

- unrelated politics
- personal medical advice
- fabricated government rules
- unsupported local numbers

Missing-data

- no competitor data
- no reliable price data

Conflicting documents

- older policy vs newer version

Numerical hallucination

- prompt tries to force the model to make up an EMI

The blueprint explicitly calls for AI tests covering unsupported questions, missing data, conflicting documents and numerical hallucination.

23. Source handling

Every retrieved document should carry:

```
document title  
source  
version  
section  
publication/effective date
```

The final answer can expose a concise source label with an expandable detail view.

24. Integration with M3/M4

M5 receives verified objects.

Never fetch raw market/finance data from an unrelated source when the same value already exists in the central backend context.

M5 should treat:

- M3 market values
- M4 financial values

as authoritative for the application session.

25. Definition of done

M5 is done when:

- scheme documents are ingested with metadata
- retrieval works
- advisor answers are grounded
- source/version is exposed
- numerical guardrails work
- missing data produces safe answers
- SWOT/risk outputs are evidence-based
- DPR is generated from the same verified context
- English/Hindi/Punjabi explanation path is feasible for MVP

26. Common mistakes

- letting the LLM calculate EMI/DSCR
- using unverified documents as official scheme policy
- removing citations/source metadata
- letting prompts invent local competitor numbers
- generating SWOT without evidence
- storing different financial numbers in prompts vs backend
- making AI the system of record